Sensor Fusion

State Estimation for GPS-Denied Navigation

Phase 7 — Research-Grade Notes

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

> Sensor fusion is where the project comes together: the disciplined combination of several imperfect, well-characterised sensors — **accelerometer, gyroscope, magnetometer, IMU, GPS, barometer** (and future **quantum magnetometer, LiDAR, camera, wheel encoder, atomic clock**) — into one estimate of position, velocity, orientation, and trajectory that is better than any sensor alone. This document builds the mathematical machinery (frames, rotations, Bayesian estimation, the Kalman family, particle filters) and the AI layer that adapts it, always from the standpoint of *operating when GPS is gone.*

Intuition → Bayesian foundations → the Kalman family (derived) → AI fusion → Trinetra-AI architecture.

Contents

1 Introduction to Sensor Fusion

1.1 What is sensor fusion?

Sensor fusion is the process of combining measurements from multiple sensors to produce an estimate of a system's state that is *more accurate, more reliable, and more complete* than any single sensor could provide. Formally, given several measurement streams $\{\mathbf{z}_t^{(1)}, \ldots, \mathbf{z}_t^{(m)}\}$ of an unknown state $\mathbf{x}_t$, fusion computes the posterior estimate $\hat{\mathbf{x}}_t = \mathbb{E}[\mathbf{x}_t \mid \mathbf{z}_{1:t}]$ together with its uncertainty.

Intuition: think of several eyewitnesses to an event, each with partial views and biases. A good detective does not pick one witness — they *weigh* each account by its credibility and cross-check them into a story more reliable than any single testimony. Sensor fusion is that detective, and the "credibility weights" are the noise covariances.

1.2 Why it exists: history and evolution

Fusion arose from the impossibility of navigating with one sensor. The **Kalman filter** (Kalman, 1960) gave the first optimal recursive solution for linear-Gaussian systems and famously enabled Apollo's inertial navigation. Nonlinearity drove the **Extended KF** (1960s, aerospace) and later the **Unscented KF** (Julier & Uhlmann, 1997) and **particle filter** (Gordon, Salmond & Smith, 1993). The modern era adds *learning*: neural-augmented filters (KalmanNet, Revach et al. 2022) and end-to-end deep fusion — the lineage Trinetra-AI extends.

1.3 Why it matters across domains

Domain	Fusion role	Trinetra-AI parallel
Robotics	IMU+odometry+LiDAR/camera → SLAM	core state estimation
Aerospace	INS+GNSS+air-data over long flights	drift-bounded navigation
Autonomous vehicles	GNSS+IMU+wheel+vision; canyon multipath	GPS-denied bridging
Defense	jam/spoof-resistant, passive nav	magnetic/terrain-aided fusion
Space (ISRO/-NASA)	star-tracker+IMU+sun sensor (no GPS)	extreme drift discipline
Quantum navigation	quantum-magnetometer + INS + AI denoising	the project's north star

Trinetra-AI *is* a sensor-fusion framework. Everything in Phases 4–6 (time series, signal processing, sensors) fed toward this moment: clean, calibrated, time-synced measurements with honest error models, ready to be combined. This phase is the engine room.

2 Why Sensor Fusion is Needed

2.1 No single sensor can navigate

Each sensor is individually insufficient, and — crucially — their weaknesses are *complementary*: where one fails, another is strong. Fusion exploits exactly this.

Sensor	Strength	Fatal weakness alone
Accelerometer	gravity reference; linear accel	double-integration drift $\propto t^2$
Gyroscope	smooth high-rate rotation	bias drift $\propto t$ in heading
Magnetometer	absolute drift-free heading; map anchor	distorted by iron/EMI
GPS	absolute position, no drift	absent indoors; multipath; jammable
Barometer	low-noise vertical	weather-driven absolute drift

2.2 The five pillars of fusion

Complementarity: sensors cover each other's frequency/failure gaps (gyro = good short-term, magnetometer = good long-term). **Redundancy**: overlapping measurements enable **fault tolerance** — a failed sensor can be detected and excluded. **Accuracy**: combining independent estimates reduces variance. **Reliability**: the system degrades gracefully instead of failing. **Bias/drift compensation**: an absolute sensor (GPS, magnetometer) continuously resets an integrating sensor's drift.

Two independent unbiased estimates of the same quantity, with variances σ_1^2, σ_2^2, fuse optimally (inverse-variance weighting) into

$$\hat{x} = \frac{\sigma_2^2\, x_1 + \sigma_1^2\, x_2}{\sigma_1^2 + \sigma_2^2}, \qquad \frac{1}{\sigma^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}.$$

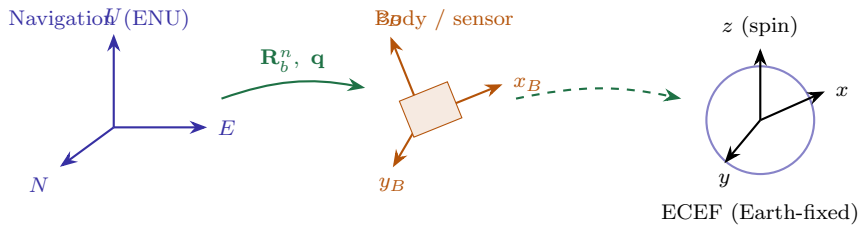
The fused variance σ^2 is *smaller than either* input — you cannot do worse by adding an independent sensor, provided you weight it correctly. This scalar result is the seed of the Kalman gain (§10), which is precisely the matrix generalisation of inverse-variance weighting.

This is the whole thesis of GPS-denied navigation: the gyro/accelerometer drift without bound, but the magnetometer (soon quantum) and barometer provide *absolute* anchors that reset that drift. Fusion is the mechanism that lets the absolute-but-sparse sensors continuously correct the smooth-but-drifting ones. No single sensor solves navigation; their *disciplined combination* does.

3 Coordinate Frames

Every measurement is expressed in *some* frame; fusing them means transforming them all into a common frame. Getting frames wrong is the single most common source of sign/axis bugs in a navigation system.

A **reference frame** is a coordinate system with an origin and axis directions. Key frames: **Body/sensor frame** (fixed to the vehicle, axes along the IMU); **Navigation/local frame** — **ENU** (East-North-Up) or **NED** (North-East-Down) — a local tangent plane; **Earth frame ECEF** (Earth-Centred Earth-Fixed, rotates with Earth); and the **global geodetic** frame (lat, lon, alt).



A vector expressed in the body frame becomes, in the navigation frame, $\mathbf{v}^n = \mathbf{R}_b^n \mathbf{v}^b$, where $\mathbf{R}_b^n \in SO(3)$ is the body-to-navigation rotation. Chaining frames composes rotations: $\mathbf{v}^e = \mathbf{R}_n^e \mathbf{R}_b^n \mathbf{v}^b$. A full pose transform (rotation + translation) uses a homogeneous 4×4 matrix $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$. ENU and NED differ by an axis swap and sign: $[E, N, U] \leftrightarrow [N, E, -U]$ — a frequent bug source.

```
import numpy as np
def enu_to_ned(v): return np.array([v[1], v[0], -v[2]])
def body_to_nav(v_b, R_nb): return R_nb @ v_b # R_nb: 3x3 rotation
```

4 Rotation Mathematics

A **rotation matrix** $\mathbf{R} \in SO(3)$ rotates vectors while preserving lengths and angles. It satisfies **orthogonality** $\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$ (so $\mathbf{R}^{-1} = \mathbf{R}^\mathsf{T}$) and $\det \mathbf{R} = +1$. Its columns are the rotated basis vectors.

Rotations about the x, y, z axes by ϕ, θ, ψ are

$$\mathbf{R}_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\phi & -s\phi \\ 0 & s\phi & c\phi \end{bmatrix}, \quad \mathbf{R}_y(\theta) = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}, \quad \mathbf{R}_z(\psi) = \begin{bmatrix} c\psi & -s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

($c = \cos, s = \sin$). Composition is *matrix product* and is *not commutative*: $\mathbf{R}_1\mathbf{R}_2 \neq \mathbf{R}_2\mathbf{R}_1$ — rotation order matters. The inverse (un-rotate) is simply the transpose, which is why rotation matrices are numerically clean.

Advantages / limits. Rotation matrices are unambiguous and compose by multiplication, but use 9 numbers for 3 degrees of freedom (redundant) and drift from orthogonality under numerical integration (requiring re-orthonormalisation) — which is one reason quaternions (§6) are preferred for propagation.

```
import numpy as np
def Rx(a): c,s=np.cos(a),np.sin(a); return np.array([[1,0,0],[0,c,-s],[0,s,c]])
def Ry(a): c,s=np.cos(a),np.sin(a); return np.array([[c,0,s],[0,1,0],[-s,0,c]])
def Rz(a): c,s=np.cos(a),np.sin(a); return np.array([[c,-s,0],[s,c,0],[0,0,1]])
def R_from_euler_zyx(roll,pitch,yaw): return Rz(yaw) @ Ry(pitch) @ Rx(roll)
```

5 Euler Angles

Euler angles describe orientation as three sequential rotations: **roll** ϕ (about x), **pitch** θ (about y), **yaw** ψ (about z). The **order** matters; aerospace commonly uses the ZYX (yaw–pitch–roll) convention: $\mathbf{R} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$.

At pitch $\theta = \pm 90°$ the ZYX composition makes the roll and yaw axes align, so one rotational degree of freedom is lost — **gimbal lock**. Mathematically the Jacobian relating Euler rates to body angular rates,

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi\sec\theta & \cos\phi\sec\theta \end{bmatrix} \boldsymbol{\omega},$$

contains $\tan\theta$ and $\sec\theta$, which *blow up* at $\theta = \pm 90°$. This singularity is why Euler angles are unsafe for full-attitude propagation.

Use & limits. Euler angles are intuitive for display and for near-level operation (roll/pitch from accelerometer, heading from magnetometer), but their gimbal-lock singularity and order-dependence make them unfit as the internal attitude state — hence quaternions.

6 Quaternions

6.1 Why quaternions, and the algebra

A **unit quaternion** $\mathbf{q} = [q_w, q_x, q_y, q_z] = q_w + q_x i + q_y j + q_z k$ (with $i^2 = j^2 = k^2 = ijk = -1$) represents a 3-D rotation with *four* numbers and *no singularities*. A rotation by angle α about unit axis $\mathbf{u}$ is $\mathbf{q} = [\cos\frac{\alpha}{2},\ \mathbf{u}\sin\frac{\alpha}{2}]$, and unit norm $\|\mathbf{q}\| = 1$ must be maintained.

Quaternion (Hamilton) product for $\mathbf{q} = [w_1, \mathbf{v}_1]$, $\mathbf{p} = [w_2, \mathbf{v}_2]$:

$$\mathbf{q} \otimes \mathbf{p} = \left[w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2,\quad w_1\mathbf{v}_2 + w_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2 \right].$$

Rotating a vector: $\mathbf{v}' = \mathbf{q} \otimes [0, \mathbf{v}] \otimes \mathbf{q}^*$ (with conjugate $\mathbf{q}^* = [w, -\mathbf{v}]$). Attitude propagation under body rate $\boldsymbol{\omega}$ obeys

$$\dot{\mathbf{q}} = \tfrac{1}{2}\,\mathbf{q} \otimes [0, \boldsymbol{\omega}],$$

integrated then renormalised. No $\tan\theta$ terms appear — quaternions are *singularity-free*, the reason robotics and aerospace use them internally.

6.2 Conversions

Quaternion→rotation matrix and quaternion↔Euler conversions are standard (and provided below); the practical pattern is: keep the *state* as a quaternion, convert to Euler only for human-readable display.

```
import numpy as np
def quat_mul(q, r):
    w0,x0,y0,z0=q; w1,x1,y1,z1=r
    return np.array([w0*w1-x0*x1-y0*y1-z0*z1, w0*x1+x0*w1+y0*z1-z0*y1,
                     w0*y1-x0*z1+y0*w1+z0*x1, w0*z1+x0*y1-y0*x1+z0*w1])
def quat_to_R(q):
    w,x,y,z=q/np.linalg.norm(q)
    return np.array([[1-2*(y*y+z*z),2*(x*y-w*z),2*(x*z+w*y)],
                     [2*(x*y+w*z),1-2*(x*x+z*z),2*(y*z-w*x)],
                     [2*(x*z-w*y),2*(y*z+w*x),1-2*(x*x+y*y)]])
def propagate_quat(q, omega, dt):
    q = q + 0.5*quat_mul(q, np.concatenate([[0.0],omega]))*dt
    return q/np.linalg.norm(q)
```

Trinetra-AI carries orientation as a **quaternion** inside the fusion state: the gyro propagates it via $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes [0, \boldsymbol{\omega}]$, and the accelerometer (gravity) and magnetometer (heading) correct it in the Kalman update. Euler angles are used only for the visualization dashboard.

7 Dead Reckoning

Dead reckoning estimates the current pose by integrating motion (velocity, or acceleration and rotation rate) forward from a known starting pose — using *no* external position reference. It is the "inertial-only" backbone that fusion later corrects.

Strapdown dead reckoning integrates the IMU:

$$\dot{\mathbf{q}} = \tfrac{1}{2}\mathbf{q} \otimes [0, \boldsymbol{\omega}], \quad \dot{\mathbf{v}} = \mathbf{R}(\mathbf{q})\,\mathbf{a}_{\text{body}} - \mathbf{g}, \quad \dot{\mathbf{p}} = \mathbf{v}.$$

Errors compound through the integrators. A gyro bias b_g gives heading error $\propto t$, which mis-rotates gravity and injects a spurious horizontal acceleration $\approx g \cdot b_g t$, producing velocity error $\propto t^2$ and position error $\propto t^3$. An accelerometer bias b_a alone gives position error $\frac{1}{2} b_a t^2$. These polynomial growth laws are unavoidable without aiding.

Why dead reckoning alone fails: its error grows without bound (up to $\propto t^3$), so within seconds-to-minutes an unaided MEMS IMU is useless for position. This is the precise problem Trinetra-AI solves: dead reckoning provides the high-rate motion estimate *between*

absolute fixes, while fusion (magnetometer/map, barometer, GPS when present) continually resets its drift. Dead reckoning is necessary but never sufficient.

```
import numpy as np
def dead_reckon(acc_b, gyro, dt, q0, v0, p0, g=np.array([0,0,9.80665])):
    q,v,p = q0.copy(), v0.copy(), p0.copy(); traj=[p.copy()]
    for a_b, w in zip(acc_b, gyro):
        q = propagate_quat(q, w, dt) # attitude
        a_nav = quat_to_R(q) @ a_b - g # remove gravity
        v = v + a_nav*dt # velocity (drifts ~t^2)
        p = p + v*dt; traj.append(p.copy()) # position (drifts ~t^3)
    return np.array(traj)
```

8 State Estimation

8.1 State, models, and covariance

The **state** $\mathbf{x}$ is the minimal set of variables that describes the system — e.g. $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \mathbf{b}_a, \mathbf{b}_g]$ (position, velocity, orientation, sensor biases). The **system (transition) model** $\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t)$ predicts how it evolves under control input $\mathbf{u}$; the **observation/measurement model** $\mathbf{z}_t = h(\mathbf{x}_t)$ links state to sensor readings. The **covariance** $\mathbf{P} = \mathbb{E}[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^\mathsf{T}]$ quantifies the estimate's uncertainty.

Estimation proceeds in two repeating steps: **prediction** (push the state forward through f, uncertainty grows) and **correction/update** (fold in a measurement through h, uncertainty shrinks). This predict–correct rhythm is common to *every* filter in this document.

Trinetra-AI's fusion state is an *error-state* formulation carrying position, velocity, quaternion, and the accelerometer/gyro biases (from Phase 6) — the biases are estimated online so the filter continuously calibrates the sensors while navigating.

9 Bayesian Estimation

9.1 Bayes' rule and recursive estimation

Given a **prior** belief $p(\mathbf{x})$ and a measurement likelihood $p(\mathbf{z} \mid \mathbf{x})$, **Bayes' rule** gives the **posterior**

$$p(\mathbf{x} \mid \mathbf{z}) = \frac{p(\mathbf{z} \mid \mathbf{x})\, p(\mathbf{x})}{p(\mathbf{z})} \;\propto\; p(\mathbf{z} \mid \mathbf{x})\, p(\mathbf{x}).$$

Read: *updated belief* $\propto$ *measurement fit* $\times$ *prior belief.*

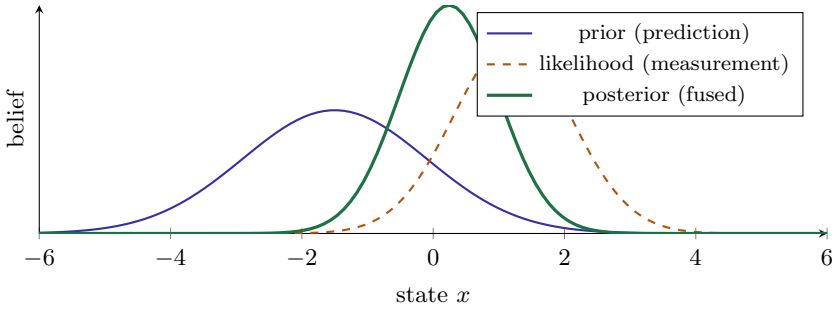
Tracking a state over time, the **predict** step propagates the prior through the motion model (Chapman–Kolmogorov):

$$p(\mathbf{x}_t \mid \mathbf{z}_{1:t-1}) = \int p(\mathbf{x}_t \mid \mathbf{x}_{t-1})\, p(\mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1})\, d\mathbf{x}_{t-1},$$

and the **update** step folds in the new measurement:

$$p(\mathbf{x}_t \mid \mathbf{z}_{1:t}) \propto p(\mathbf{z}_t \mid \mathbf{x}_t)\, p(\mathbf{x}_t \mid \mathbf{z}_{1:t-1}).$$

Every recursive filter is an implementation of these two lines. The Kalman filter is the exact solution when everything is linear-Gaussian; the particle filter approximates it with samples when it is not.



The posterior is *narrower* (more certain) than either the prior or the likelihood, and sits between them — pulled toward whichever is more confident. This picture *is* the Kalman update: multiplying two Gaussians yields a sharper Gaussian.

This Bayesian view unifies the whole fusion stack: predict = prior from the motion model, update = posterior after a sensor measurement. Seeing KF, EKF, UKF, and particle filters as *one* Bayesian family (differing only in how they represent and propagate the belief) is the key conceptual payoff of this phase.

10 Kalman Filter

The Kalman filter (KF) is the optimal recursive estimator for *linear* systems with *Gaussian* noise — and the beating heart of Trinetra-AI's fusion core.

10.1 Model and intuition

The KF assumes a linear-Gaussian state-space model

$$\mathbf{x}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q}), \qquad \mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R}),$$

with **state-transition** $\mathbf{F}$, **control** $\mathbf{B}$, **observation** $\mathbf{H}$, **process noise** covariance $\mathbf{Q}$, and **measurement noise** covariance $\mathbf{R}$. It maintains the state estimate $\hat{\mathbf{x}}$ and covariance $\mathbf{P}$.

Intuition: the KF keeps a single Gaussian belief. Each cycle it *predicts* forward (belief drifts and widens) then *corrects* with a measurement (belief snaps back and narrows), trading the two off by the **Kalman gain** — how much to trust the new measurement versus the prediction.

10.2 The equations

Predict (project state & covariance forward):

$$\hat{\mathbf{x}}^- = \mathbf{F}\hat{\mathbf{x}} + \mathbf{B}\mathbf{u}, \qquad \mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q}.$$

Update (fold in measurement $\mathbf{z}$): the **innovation** (residual) and its covariance are

$$\mathbf{y} = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-, \qquad \mathbf{S} = \mathbf{H}\mathbf{P}^-\mathbf{H}^\mathsf{T} + \mathbf{R}.$$

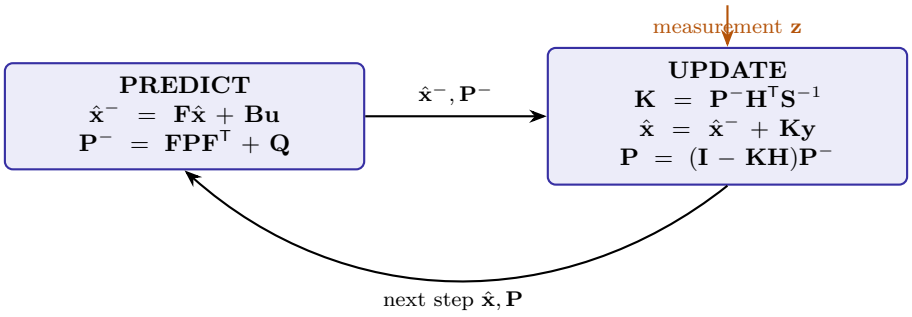
The **Kalman gain**, covariance-weighted optimal blend, is

$$\boxed{\mathbf{K} = \mathbf{P}^-\mathbf{H}^\mathsf{T}\mathbf{S}^{-1}}$$

and the corrected state and covariance are

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + \mathbf{K}\mathbf{y}, \qquad \mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-.$$

The gain minimises the posterior error covariance. Writing the updated covariance for a generic gain $\mathbf{K}$, $\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-(\mathbf{I} - \mathbf{K}\mathbf{H})^\mathsf{T} + \mathbf{K}\mathbf{R}\mathbf{K}^\mathsf{T}$, and minimising its trace, $\partial\,\mathrm{tr}(\mathbf{P})/\partial\mathbf{K} = 0$, gives $\mathbf{K} = \mathbf{P}^-\mathbf{H}^\mathsf{T}(\mathbf{H}\mathbf{P}^-\mathbf{H}^\mathsf{T} + \mathbf{R})^{-1}$. Note the two limits: with *tiny* measurement noise ($\mathbf{R} \to 0$), $\mathbf{K} \to \mathbf{H}^{-1}$ (trust the measurement); with *huge* $\mathbf{R}$, $\mathbf{K} \to 0$ (ignore it, keep the prediction). This is exactly inverse-variance weighting (§2) in matrix form.



10.3 Worked numerical example (1-D constant velocity)

Track position & velocity, $\mathbf{x} = [p, v]^\mathsf{T}$, with $\Delta t = 1$, so $\mathbf{F} = \left[\begin{smallmatrix}1 & 1\\ 0 & 1\end{smallmatrix}\right]$, $\mathbf{H} = [1\ 0]$ (we measure position). Start $\hat{\mathbf{x}} = [0, 1]^\mathsf{T}$, $\mathbf{P} = \left[\begin{smallmatrix}1 & 0\\ 0 & 1\end{smallmatrix}\right]$, $\mathbf{Q} = \left[\begin{smallmatrix}0.1 & 0\\ 0 & 0.1\end{smallmatrix}\right]$, $R = 0.5$, and a measurement $z = 1.2$. *Predict:* $\hat{\mathbf{x}}^- = [1, 1]^\mathsf{T}$, $\mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q} = \left[\begin{smallmatrix}2.1 & 1\\ 1 & 1.1\end{smallmatrix}\right]$. *Update:* $y = 1.2 - 1 = 0.2$, $S = P^-_{11} + R = 2.6$, $\mathbf{K} = \left[\begin{smallmatrix}2.1\\ 1\end{smallmatrix}\right]/2.6 = \left[\begin{smallmatrix}0.808\\ 0.385\end{smallmatrix}\right]$, so $\hat{\mathbf{x}} = [1, 1]^\mathsf{T} + \mathbf{K}(0.2) = [1.162,\ 1.077]^\mathsf{T}$ and the position variance drops from 2.1 to $(1 - 0.808) \cdot 2.1 \approx 0.40$. The measurement pulled the

estimate toward 1.2 and *sharpened* the belief.

```
import numpy as np
from filterpy.kalman import KalmanFilter
kf = KalmanFilter(dim_x=2, dim_z=1)
kf.F = np.array([[1.,1.],[0.,1.]]); kf.H = np.array([[1.,0.]])
kf.P *= 1.0; kf.Q = np.eye(2)*0.1; kf.R = np.array([[0.5]])
kf.x = np.array([[0.],[1.]])
kf.predict(); kf.update(np.array([[1.2]]))
print(kf.x.ravel(), kf.P.diagonal()) # fused state and its variances
```

KF in Trinetra-AI: the filter fuses the IMU-driven prediction with magnetometer heading, barometric altitude, and GPS (when present). $\mathbf{Q}$ comes from the Allan-variance noise budget (Phase 6); $\mathbf{R}$ from each sensor's measured noise, adapted online by the AI confidence module. The innovation $\mathbf{y}$ doubles as the drift/anomaly signal fed back to the AI modules. The plain KF is the baseline; real navigation is nonlinear, so we extend it next.

11 Extended Kalman Filter (EKF)

The **EKF** handles *nonlinear* models $\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t)$, $\mathbf{z}_t = h(\mathbf{x}_t)$ by *linearising* them about the current estimate using first-order Taylor expansion. The linear KF is then applied to the linearised system.

Replace $\mathbf{F}, \mathbf{H}$ with the Jacobians evaluated at the current estimate:

$$\mathbf{F} = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}}, \qquad \mathbf{H} = \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}^-}.$$

Predict uses the *nonlinear* f for the mean but the Jacobian for the covariance:

$$\hat{\mathbf{x}}^- = f(\hat{\mathbf{x}}, \mathbf{u}), \quad \mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^\mathsf{T} + \mathbf{Q};$$

update uses $\mathbf{y} = \mathbf{z} - h(\hat{\mathbf{x}}^-)$ with the same gain formula as the KF but the Jacobian $\mathbf{H}$. *The "Extended" is exactly this Jacobian step.*

Applications / limits. The EKF is the workhorse of INS/GNSS integration and IMU orientation fusion (Sabatini-style quaternion EKF). Its weakness: linearisation error when f, h are strongly nonlinear or the uncertainty is large, which can bias the estimate or diverge; and hand-deriving Jacobians is error-prone.

```
import numpy as np
def ekf_step(x, P, u, z, f, h, F_jac, H_jac, Q, R):
    x_pred = f(x, u); F = F_jac(x, u) # predict
    P_pred = F @ P @ F.T + Q
    H = H_jac(x_pred) # linearise measurement
    y = z - h(x_pred); S = H @ P_pred @ H.T + R
    K = P_pred @ H.T @ np.linalg.inv(S) # gain
    x_new = x_pred + K @ y
    P_new = (np.eye(len(x)) - K @ H) @ P_pred
```

```
    return x_new, P_new
```

12 Unscented Kalman Filter (UKF)

The **UKF** avoids Jacobians by propagating a small, deterministic set of **sigma points** through the *true* nonlinear function and recomputing their mean and covariance — the **unscented transform**. It captures the posterior mean/covariance to *second* order (vs. the EKF's first), often more accurately and with no derivatives.

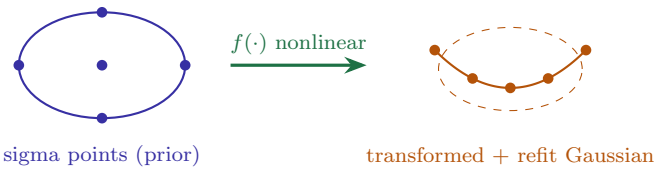
For an n-dimensional state, generate $2n{+}1$ sigma points from the mean $\hat{\mathbf{x}}$ and covariance $\mathbf{P}$:

$$\boldsymbol{\chi}_0 = \hat{\mathbf{x}}, \qquad \boldsymbol{\chi}_i = \hat{\mathbf{x}} \pm \left(\sqrt{(n+\lambda)\mathbf{P}}\right)_i,$$

with scaling $\lambda = \alpha^2(n+\kappa) - n$. Propagate each through f (or h), then recombine with weights W_i^m, W_i^c:

$$\hat{\mathbf{x}}^- = \sum_i W_i^m f(\boldsymbol{\chi}_i), \qquad \mathbf{P}^- = \sum_i W_i^c \big(f(\boldsymbol{\chi}_i) - \hat{\mathbf{x}}^-\big)\big(f(\boldsymbol{\chi}_i) - \hat{\mathbf{x}}^-\big)^{\mathsf{T}} + \mathbf{Q}.$$

No linearisation, no Jacobian — the nonlinearity is sampled, not approximated.



	EKF	UKF
Nonlinearity	1st-order Taylor (Jacobian)	sigma points through true f (2nd order)
Derivatives	required (error-prone)	**none**
Accuracy	degrades if strongly nonlinear	generally better
Cost	low	moderate ($2n{+}1$ evaluations)

```
from filterpy.kalman import UnscentedKalmanFilter, MerweScaledSigmaPoints
import numpy as np
pts = MerweScaledSigmaPoints(n=2, alpha=1e-3, beta=2., kappa=0.)
ukf = UnscentedKalmanFilter(dim_x=2, dim_z=1, dt=1.0,
                            fx=lambda x,dt:[x[0]+x[1]*dt, x[1]],
                            hx=lambda x:[x[0]], points=pts)
ukf.Q=np.eye(2)*0.1; ukf.R=np.array([[0.5]]); ukf.x=np.array([0.,1.]); ukf.P*=1.
ukf.predict(); ukf.update([1.2]); print(ukf.x)
```

13 Particle Filter

A **particle filter (PF)** represents the belief *non-parametrically* by a swarm of weighted samples (**particles**) $\{\mathbf{x}^{(i)}, w^{(i)}\}$. It handles arbitrary *non-Gaussian, multi-modal* distributions — exactly the case for magnetic-map matching, where several map locations may fit the measurement.

Predict: propagate each particle through the motion model (with noise), $\mathbf{x}_t^{(i)} \sim p(\mathbf{x}_t \mid \mathbf{x}_{t-1}^{(i)})$. **Update (importance weighting):** weight by the measurement likelihood,

$$w_t^{(i)} \propto w_{t-1}^{(i)}\, p(\mathbf{z}_t \mid \mathbf{x}_t^{(i)}), \qquad \textstyle\sum_i w_t^{(i)} = 1.$$

Resample: when the effective sample size $N_{\text{eff}} = 1/\sum_i (w^{(i)})^2$ falls below a threshold, draw a new particle set with probability $\propto w^{(i)}$ (systematic resampling) to combat *degeneracy* (all weight on one particle). The state estimate is the weighted mean $\hat{\mathbf{x}} = \sum_i w^{(i)}\mathbf{x}^{(i)}$.

Applications / limits. PFs power Monte-Carlo localisation, FastSLAM, and magnetic/terrain map-matching. Strengths: no linearisation, handles multimodality. Weaknesses: cost grows with state dimension (*curse of dimensionality*) and particle depletion — so PFs are used in Trinetra-AI for the low-dimensional *map-matching* sub-problem, not the full high-dimensional inertial state.

```
import numpy as np
def particle_filter_step(P, w, u, z, motion, likelihood, resample_thresh=0.5):
    P = motion(P, u) # predict (with noise)
    w = w * likelihood(z, P); w /= w.sum() # importance update
    x_est = np.average(P, axis=0, weights=w) # weighted-mean estimate
    if 1.0/np.sum(w**2) < resample_thresh*len(w): # resample if degenerate
        idx = np.random.choice(len(w), len(w), p=w)
        P, w = P[idx], np.ones(len(w))/len(w)
    return P, w, x_est
```

Trinetra-AI uses a **particle filter for magnetic-map matching**: each particle is a candidate position, weighted by how well the measured magnetic field matches the map at that location. This resolves the multi-modal ambiguity (several places share a field value) that a Kalman filter cannot represent, then hands a position fix to the main EKF/UKF.

14 Comparison of Fusion Algorithms

Algorithm	Principle	Nonlin.	Cost	Best use in Trinetra-AI
Kalman (KF)	optimal linear-Gaussian	no	low	linear baseline
EKF	Jacobian linearisation	yes (1st)	low	main INS/GNSS/mag fusion
UKF	sigma points / unscented	yes (2nd)	med	strongly-nonlinear attitude
Particle filter	weighted samples	any	high	magnetic-map matching
Complementary	freq.-split blend	mild	very low	cheap attitude (edge)
Mahony	passive complementary on $SO(3)$	mild	low	attitude backup
Madgwick	gradient-descent MARG fusion	mild	low	lightweight orientation
Federated	bank of local filters + master	yes	med	modular multi-sensor
Adaptive KF	online-tuned $\mathbf{Q}, \mathbf{R}$	yes	med	changing conditions
Deep-learning fusion	learned end-to-end estimator	any	high	learned displacement (RoNIN/TLIO)
Transformer fusion	attention over sensor streams	any	high	long-range multi-sensor

The simplest fusion of gyro and accelerometer for tilt:

$$\theta_t = \alpha\,(\theta_{t-1} + \omega\,\Delta t) + (1-\alpha)\,\theta_{\text{accel}},$$

which *high-passes* the gyro (good short-term, drops slow drift) and *low-passes* the accelerometer (good long-term, drops vibration). It is a fixed-gain special case of the Kalman filter and is ideal when compute is scarce.

Madgwick & Mahony filters are efficient nonlinear complementary filters on the rotation group, popular on microcontrollers; **federated** filters run per-sensor local filters fused by a master (fault-isolating); **adaptive** KFs tune $\mathbf{Q}, \mathbf{R}$ online. **Learned** fusion (deep/transformer) replaces or augments these — the subject of §16.

15 Sensor Fusion Architectures

Fusion can happen at three *levels*: **low-level (raw/data)** — combine raw measurements directly (tightly-coupled, most accurate, most complex); **feature-level** — combine extracted features (e.g. learned displacement); and **decision-level** — combine per-sensor estimates/decisions (loosely-coupled, modular, robust). Orthogonally, the *topology* may be **centralised** (one filter sees all), **distributed/federated** (local filters + fusion), or **hierarchical/hybrid**.

Architecture	Trade-off	Trinetra-AI use
Low-level (tight)	most accurate; complex, brittle to faults	IMU+GNSS raw pseudoranges
Feature-level	flexible; needs good features	learned displacement + mag features
Decision-level (loose)	modular, fault-robust; less accurate	fallback & redundancy
Centralised	optimal; single point of failure	main EKF/UKF core
Distributed/federated	fault-isolating, scalable	per-sensor health + master fuse
Hybrid	balances accuracy & robustness	**chosen design**

Trinetra-AI uses a **hybrid** architecture: a centralised error-state EKF/UKF core for tightly-coupled IMU/mag/baro/GPS fusion, wrapped by decision-level health/anomaly modules that can isolate a faulty sensor, and a feature-level path where learned displacement and magnetic-map matching feed the core. This balances the accuracy of tight coupling with the fault-tolerance of loose coupling.

16 AI in Sensor Fusion

Classical filters are optimal only under assumptions (linearity, known Gaussian noise) that real navigation violates. AI relaxes these — the core research thrust of Trinetra-AI.

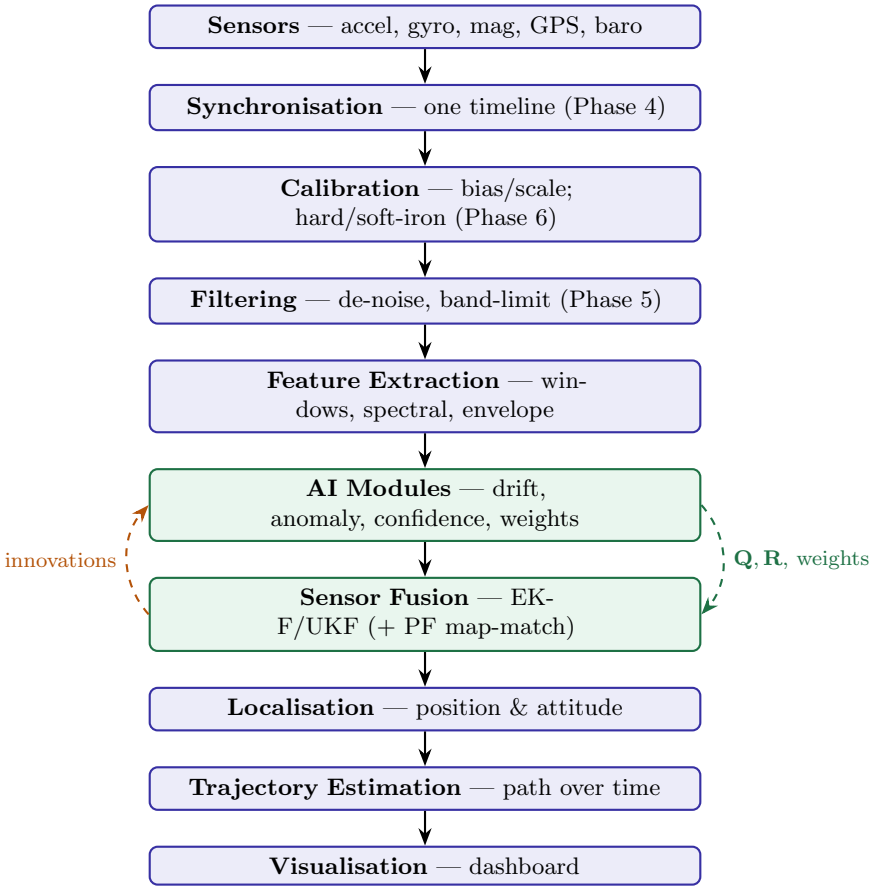
AI technique	Role in fusion / Trinetra-AI module
ML/DL fusion	learn the measurement or displacement model from data (RoNIN/TLIO)
Attention / Transformers	weight sensor streams by context; long-range temporal fusion
Graph neural networks	model sensors as a graph; relational fusion
Neural Kalman filters	learn the Kalman gain / noise (KalmanNet, Revach et al. 2022) — hybrid model-based + data-driven
Adaptive sensor weighting	network outputs per-sensor weights → adaptive $\mathbf{R}$
Confidence estimation	predict per-measurement uncertainty (TLIO-style) → $\mathbf{R}$
Fault & anomaly detection	flag bad sensors via reconstruction/forecast residual
Drift prediction	forecast bias ahead of time and pre-correct (Phase 4)
Sensor-health monitoring	track degradation; trigger reweighting/fallback

KalmanNet keeps the KF's predict/update *structure* but replaces the gain computation $\mathbf{K} = \mathbf{P}^{-}\mathbf{H}^{\mathsf{T}}\mathbf{S}^{-1}$ — which requires knowing $\mathbf{Q}, \mathbf{R}$ and the model — with a recurrent network that *learns* the gain from data. This retains the filter's interpretability and data-efficiency while coping with unknown noise statistics and model mismatch. It is the template for Trinetra-AI's "intelligent" fusion: physics-structured, data-adapted.

Where AI fits in Trinetra-AI: the AI modules sit *around* the Kalman core, not instead of it. They (i) supply learned measurements (displacement, map-match); (ii) set the filter's adaptive $\mathbf{Q}, \mathbf{R}$ and per-sensor weights via confidence/health estimates; (iii) pre-correct drift; and (iv) consume the filter's innovations for anomaly detection — the closed loop that makes

> the fusion "intelligent" while keeping the guarantees and interpretability of the classical filter.

17 Multi-Sensor Navigation Pipeline



Each stage was built in earlier phases; fusion is where they converge. The dashed loops are the intelligence: AI sets the filter's noise/weights forward, the filter's innovations feed anomaly/health backward.

18 Datasets

Real, verified; confirm licence/download on the official page. Difficulty is relative to a solo project.

Dataset	Sensors	Ground truth	Diff.	Best for
EuRoC MAV	IMU + stereo	Vicon/Leica	Med	drone VIO fusion (Burri et al., 2016)
KITTI	IMU+GPS/RTK+LiDAR+cam	RTK-GPS/INS	Med	autonomous-driving fusion (Geiger et al.)
TUM VI	IMU + stereo	partial MoCap	Med	visual-inertial (Schubert et al., 2018)
RoNIN	IMU (phone)	3D traj.	Med	learned inertial (Herath et al., 2020)
TLIO	IMU (headset)	VIO	Med	learned displacement + EKF (Liu et al., 2020)
OxIOD	IMU (accel/gyro/mag)	Vicon/VI	Easy–Med	inertial odometry (Chen et al.)
comma2k19	IMU+GNSS+CAN	GNSS+fusion	Med	GPS/INS driving fusion
KAIST Urban	IMU+GPS+LiDAR	fused	Hard	urban-canyon fusion
MagPIE	IMU + magnetometer	cm-level	Med–Hard	magnetic map-matching (Hanley et al., 2017)
DAF-MIT AIA MagNav	aeromagnetic + INS	flight ref.	Hard	magnetic/quantum navigation (Gnadt et al., 2023)

Recommended: EuRoC or KITTI to build/validate the EKF fusion core against strong ground truth, plus MagPIE + DAF-MIT AIA for the magnetic map-matching (quantum-navigation) path — together covering both halves of Trinetra-AI.

19 Research Papers

Verified, correctly attributed. Priority: ⋆⋆⋆ read first, ⋆⋆ core, ⋆ as needed.

19.1 Foundational — estimation & fusion

Work	Authors, year	Contribution	Pri.
A New Approach to Linear Filtering & Prediction	Kalman, 1960	the Kalman filter	⋆⋆⋆
A New Extension of the KF to Nonlinear Systems (UKF)	Julier & Uhlmann, 1997	unscented transform	⋆⋆⋆
The Unscented KF for Nonlinear Estimation	Wan & van der Merwe, 2000	practical UKF	⋆⋆
Novel approach to nonlinear/non-Gaussian Bayesian state estimation (particle filter)	Gordon, Salmond & Smith, 1993	bootstrap PF	⋆⋆⋆
Probabilistic Robotics	Thrun, Burgard & Fox, 2005	KF/PF/SLAM reference	⋆⋆⋆
Nonlinear Complementary Filters on $SO(3)$ (Mahony)	Mahony, Hamel & Pflimlin, 2008	attitude fusion	⋆⋆
Estimation of IMU/MARG Orientation (Madgwick)	Madgwick et al., ICORR 2011	lightweight orientation	⋆⋆

19.2 Learned fusion, neural-Kalman & navigation

Paper	Authors, year	Helps Trinetra-AI	Pri.
KalmanNet: NN-Aided Kalman Filtering	Revach et al., IEEE TSP 2022	learned gain; hybrid model+data fusion	⋆⋆⋆
TLIO: Tight Learned Inertial Odometry	Liu et al., RA-L 2020	learned displacement + uncertainty in EKF	⋆⋆⋆
RoNIN	Herath et al., ICRA 2020	neural inertial nav + benchmark	⋆⋆⋆
IONet	Chen et al., AAAI 2018	first DNN inertial odometry	⋆⋆
VINet: Visual-Inertial Odometry as Seq2Seq	Clark et al., AAAI 2017	end-to-end deep VIO fusion	⋆⋆
Denoising IMU Gyroscopes with DL	Brossard et al., RA-L 2020	learned de-noising pre-fusion	⋆⋆
DAF-MIT AIA MagNav (dataset)	Gnadt et al., 2023	magnetic-navigation benchmark	⋆⋆
Quantum-assured magnetic navigation	Muradoglu et al., 2025 (arXiv:2504.08167)	quantum-magnetometer nav vs strategic INS	⋆⋆

Reading path: Kalman (1960) → Julier/Uhlmann (UKF) → Gordon et al. (PF) → Thrun et al. (unifying text) for the classical core; then KalmanNet → TLIO → RoNIN for the learned-fusion frontier Trinetra-AI extends. **Citation hygiene:** verify every DOI/venue on the official page; cite quantum-nav industry milestones (Boeing 2024, SandboxAQ AQNav, Q-CTRL Ironstone Opal) from primary sources.

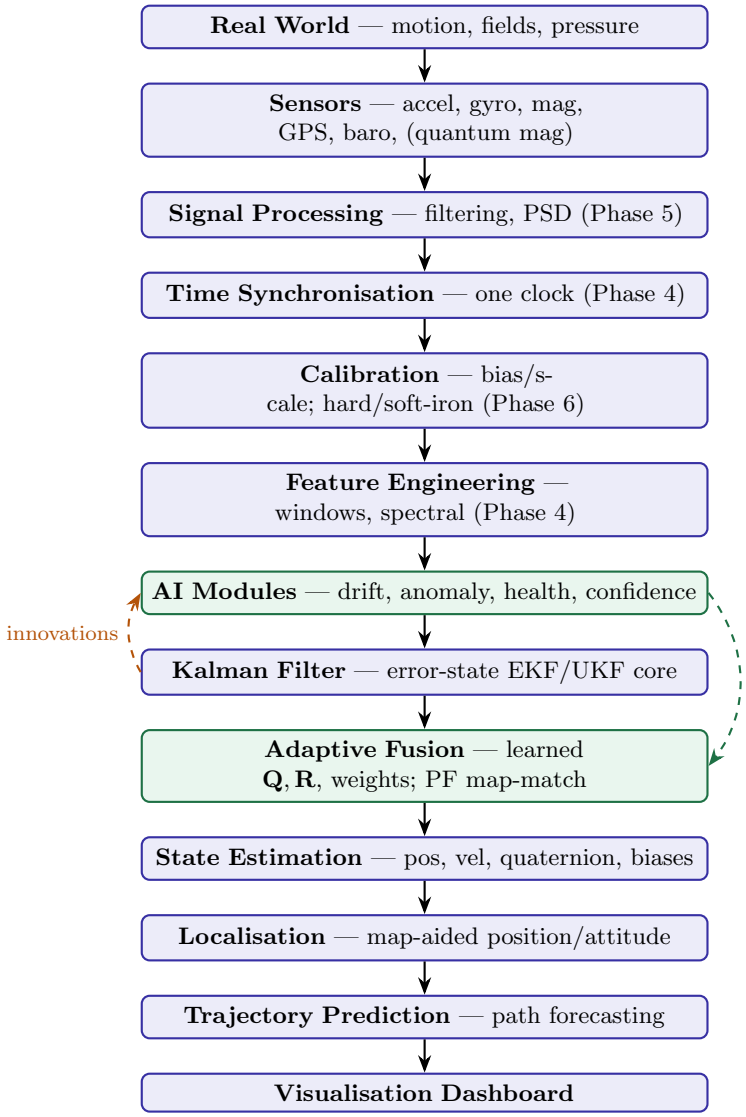
20 Practical Implementation

Notebook	Contents	Expected output
`01_frames_rotations.ipynb`	frames, **R**, Euler, quaternion	verified transforms
`02_deadreckoning.ipynb`	strapdown integration	drifting baseline trajectory
`03_kf_1d.ipynb`	KF constant-velocity (FilterPy)	fused position/velocity
`04_ekf_imu.ipynb`	quaternion EKF (IMU+mag)	attitude estimate vs truth
`05_ukf.ipynb`	UKF sigma-point fusion	nonlinear state estimate
`06_particle_mapmatch.ipynb`	PF magnetic map-matching	position fix from field
`07_full_fusion.ipynb`	EKF core + AI weights + baro/GPS	full navigation state
`08_eval_dashboard.ipynb`	ATE/RTE, NEES, plots	metrics + trajectory dashboard

Evaluation metrics. ATE/RTE (trajectory error), position RMSE, orientation error (deg), drift rate (m per 100 m), and — for filter consistency — the **NEES** (normalised estimation error squared) and **NIS** (normalised innovation squared), which check that the reported covariance is neither over- nor under-confident. **Visualisation:** trajectory vs ground truth, covariance ellipses, innovation sequences, per-sensor weights over time.

Debugging. If the filter diverges: check frame conventions (ENU vs NED, sign errors); verify $\mathbf{Q}, \mathbf{R}$ scale (too-small $\mathbf{R}$ over-trusts noisy measurements); confirm timestamps/sync; watch the innovation — a biased innovation means a model or calibration error. **Best practices:** start from a well-tuned classical filter before adding AI; unit-test transforms; use `filtfilt`/error-state formulation; validate consistency with NEES/NIS; always keep a dead-reckoning-only baseline for comparison. **Common mistakes:** un-normalised quaternions; gating GPS multipath omitted; $\mathbf{R}$ estimated on moving data; mismatched units; forgetting gravity removal.

21 Trinetra-AI Architecture



Every block. Signal processing and sync/calibration deliver clean, aligned, bias-free measurements; feature engineering feeds the *AI modules*, which produce learned measurements, confidence, and drift pre-correction; the *Kalman core* fuses everything; *adaptive fusion* lets the AI set $\mathbf{Q}, \mathbf{R}$ and per-sensor weights and runs particle-filter map-matching; the resulting *state* drives localisation, trajectory prediction, and the dashboard. The dashed loops close the intelligence: AI tunes the filter forward; innovations feed anomaly/health backward.

22 Research Contribution

Research gaps this project can target (M.Tech-thesis-scale): (1) *Learned adaptive* $\mathbf{Q}, \mathbf{R}$: a network that sets the filter's noise online from signal features, benchmarked against fixed and innovation-adaptive baselines. (2) *Uncertainty-aware learned measurements*: extend TLIO-style displacement with calibrated confidence feeding the EKF, measured on GPS-outage drift reduction. (3) *Neural-Kalman for magnetic navigation*: a KalmanNet-

style learned gain for mag-map matching under platform interference. (4) *Robust fusion under sensor faults*: learned anomaly gating vs. classical chi-square, on injected faults.

Which generative/learning paradigms help fusion? *Transformers* — strong for long-range multi-sensor temporal fusion and learned gains (clear fit). *GANs/VAEs* — useful for data augmentation and denoising (synthesise rare maneuvers, fill missing modalities), and for anomaly detection via reconstruction. *Diffusion models* — emerging for probabilistic state/trajectory generation and imputation (promising but heavier). *Reinforcement learning* — for *policy-level* decisions (when to trust/switch sensors, active sensing), less for the estimator itself. A defensible thesis focuses on **transformer-based adaptive weighting** or **neural-Kalman** contributions with rigorous classical baselines and ablations.

23 Interview & Research Preparation

23.1 60 interview questions

1. Define sensor fusion.
2. Why can't one sensor navigate?
3. Complementarity vs redundancy.
4. How does fusing reduce variance?
5. Inverse-variance weighting.
6. Body vs navigation frame.
7. ENU vs NED.
8. ECEF vs geodetic.
9. Transform a vector between frames.
10. Rotation-matrix properties.
11. Why is $\mathbf{R}^{-1} = \mathbf{R}^{\mathsf{T}}$?
12. Compose two rotations.
13. Elementary rotation matrices.
14. Euler angle order.
15. Gimbal lock (math).
16. Euler-rate Jacobian singularity.
17. Why quaternions?
18. Quaternion from axis-angle.
19. Quaternion multiplication.
20. Rotate a vector by a quaternion.
21. Quaternion kinematics.
22. Quaternion vs Euler vs matrix.
23. Dead reckoning concept.
24. Strapdown mechanization.
25. Why DR error grows $\propto t^3$.
26. What is the state vector?

27. System vs observation model.
28. Predict vs update.
29. What is covariance $\mathbf{P}$?
30. Bayes rule.
31. Recursive Bayes filter.
32. Predict integral (Chapman–Kolmogorov).
33. Multiplying two Gaussians.
34. KF assumptions.
35. KF predict equations.
36. Innovation & its covariance.
37. Kalman gain formula.
38. Gain limits ($\mathbf{R} \to 0, \infty$).
39. Covariance update.
40. Why KF is optimal (linear-Gaussian).
41. Why EKF exists.
42. EKF Jacobians.
43. EKF limitations.
44. Why UKF exists.
45. Sigma points.
46. Unscented transform.
47. UKF vs EKF.
48. Particle-filter particles & weights.
49. Importance sampling.
50. Resampling & degeneracy.
51. N_{eff}.
52. When PF over KF.
53. Complementary filter.
54. Madgwick/Mahony.
55. Federated filter.
56. Adaptive KF.
57. Tight vs loose coupling.
58. Centralised vs distributed.
59. NEES/NIS consistency.
60. Where AI fits in fusion.

23.2 40 viva questions

1. Derive the Kalman gain.
2. Derive the covariance update.
3. Show the gain is inverse-variance weighting.

4. Work a 1-D KF cycle by hand.
5. Derive EKF from the KF.
6. Derive sigma-point weights.
7. Show UKF captures 2nd-order accuracy.
8. Derive the PF weight update.
9. Explain systematic resampling.
10. Derive quaternion kinematics.
11. Show Euler gimbal lock analytically.
12. Set up an IMU error-state EKF.
13. Choose **Q** from Allan variance.
14. Choose **R** from noise.
15. Explain NEES and what over/under-confidence looks like.
16. Why does an EKF diverge?
17. Tune a diverging filter.
18. Fuse mag heading into an attitude filter.
19. Fuse baro altitude.
20. Gate GPS multipath.
21. Tight vs loose coupling trade-off.
22. Design a federated filter.
23. When is a PF necessary?
24. Curse of dimensionality in PFs.
25. Complementary filter as a KF special case.
26. Derive the complementary blend.
27. Explain KalmanNet.
28. Where do learned measurements enter the filter?
29. Confidence $\rightarrow$ adaptive **R**.
30. Innovation as an anomaly signal.
31. Fault detection via chi-square.
32. Prove covariance stays symmetric positive-definite.
33. Joseph-form update & why.
34. Observability of the state.
35. Effect of sync error on fusion.
36. Frame-convention bugs.
37. Quaternion normalisation drift.
38. Validate against ground truth.
39. Ablate an AI module.
40. Explain the full architecture.

23.3 30 research questions

1. Learned vs innovation-adaptive $\mathbf{Q}, \mathbf{R}$: measurable gains?
2. Calibrated uncertainty from learned measurements.
3. Neural-Kalman for mag navigation under interference.
4. Transformer fusion vs EKF on long outages.
5. GNN relational fusion of heterogeneous sensors.
6. RL for active sensor selection.
7. Diffusion models for trajectory imputation.
8. GAN augmentation for rare maneuvers.
9. Robustness to injected sensor faults.
10. Consistency (NEES) of learned filters.
11. Generalisation across platforms (cf. MagNav).
12. Quantum-magnetometer noise modelling in fusion.
13. PF vs learned map-matching accuracy.
14. Tight vs loose coupling under GPS-denial.
15. Edge-deployable fusion on Jetson.
16. Online calibration inside the filter.
17. Fusion stability when AI is wrong.
18. Sync-error compensation learning.
19. Uncertainty propagation through learned modules.
20. Federated fusion fault isolation.
21. Adaptive weighting vs fixed on real data.
22. Drift-prediction impact on ATE.
23. Multi-hypothesis handling (PF vs GSF).
24. Observability with intermittent aiding.
25. Data efficiency of hybrid vs end-to-end.
26. Domain gap sim→real.
27. Confidence calibration (reliability diagrams).
28. Long-horizon trajectory forecasting.
29. Magnetic-map resolution vs accuracy.
30. Benchmark design for GPS-denied fusion.

23.4 20 coding questions

1. Implement frame transforms (ENU/NED/body).
2. Quaternion multiply & rotate.
3. Quaternion↔Euler↔matrix.
4. Strapdown dead reckoning.
5. 1-D KF from scratch.
6. KF with FilterPy.

7. Quaternion EKF for IMU+mag.
8. UKF with Merwe sigma points.
9. Particle filter for 1-D localisation.
10. Magnetic map-matching PF.
11. Complementary filter for tilt.
12. Madgwick filter.
13. Adaptive $\mathbf{R}$ from a confidence signal.
14. GPS chi-square gate.
15. Compute NEES/NIS over a run.
16. Plot covariance ellipses.
17. Fuse baro altitude into the state.
18. Inject a sensor fault and detect it.
19. Compute ATE/RTE vs ground truth.
20. Build the fusion dashboard.

24 Summary, Glossary & Roadmap

24.1 Summary

Sensor fusion is the disciplined combination of imperfect sensors into one state estimate. Its foundations are coordinate frames and rotations (quaternions internally), dead reckoning (the drifting inertial backbone), and Bayesian estimation (predict = prior, update = posterior). The Kalman family implements this: the KF (optimal linear-Gaussian), the EKF (Jacobian linearisation), the UKF (sigma points), and the particle filter (samples for non-Gaussian, multi-modal problems like magnetic map-matching). AI wraps the classical core — supplying learned measurements, adaptive noise/weights, drift pre-correction, and anomaly detection — making fusion *intelligent* while keeping the interpretability and guarantees of the filter. This is the engine of Trinetra-AI.

24.2 Glossary

ATE/RTE — absolute/relative trajectory error. **Covariance P** — uncertainty of the estimate. **Dead reckoning** — pose by integrating motion from a known start. **EKF/UKF** — extended / unscented Kalman filter. **Innovation** — measurement minus prediction, $\mathbf{z}-\mathbf{H}\hat{\mathbf{x}}^{-}$. **Kalman gain** — optimal predict-vs-measurement blend. **NEES/NIS** — consistency checks on covariance/innovation. **Particle filter** — sample-based non-Gaussian estimator. **Process/measurement noise $\mathbf{Q}, \mathbf{R}$** — model / sensor uncertainty. **Quaternion** — 4-number singularity-free attitude. **Sigma points** — deterministic samples for the unscented transform. **Tight/loose coupling** — raw-measurement / estimate-level fusion.

24.3 Roadmap: Phase 7 → Phase 8, Phase 9 & the Trinetra-AI architecture

Next phase	What this phase hands it
Phase 8 — Navigation Systems	the fused state (position, velocity, attitude) and the map-aided localisation that define GPS-denied operation
Phase 9 — AI Models	the hooks where learned modules plug in (adaptive $\mathbf{Q}, \mathbf{R}$, learned measurements, neural-Kalman gain, anomaly/health)
Trinetra-AI architecture	the complete estimation core around which signal processing, sensors, and AI are organised

You now have the estimation engine. Phase 8 turns fused state into a full navigation system (dead-reckoning + map aiding + trajectory), and Phase 9 deepens the AI modules that make the fusion adaptive. Everything from Phases 4–6 (time series, signal processing, sensors) feeds this core, and everything after builds on it — this is the centre of Trinetra-AI.

End of Phase 7 notes. Next: Phase 8 (Navigation Systems) → Phase 9 (AI Models) → the final Trinetra-AI architecture.